Architectural Design Document: PKI Audit & Compliance Platform

1. Project Scope & Objective

This platform is designed to provide automated, continuous auditing of enterprise Public Key Infrastructure (PKI) environments, specifically engineered to ensure compliance with the NIS2 directive.
The system operates in two distinct phases:

1. **Deterministic Auditing (Core Backend):** Mathematically parsing X.509 certificates to verify cryptographic strength (e.g., RSA key sizes $\ge$ 2048), signature algorithms, and validity periods.
2. **Heuristic Analysis (AI Layer):** Utilizing a Python-based Retrieval-Augmented Generation (RAG) prototype to analyze audit logs against unstructured internal security policies, detect anomalies, and generate remediation strategies.

2. Current Architecture (The Prototype Stack)

The current architecture is optimized for rapid development, zero-infrastructure overhead, and strict separation of concerns.

2.1 The Full-Stack Components

- **Presentation Layer:** React & TypeScript (Provides the visual command center and compliance dashboards).
- **API & Service Layer:** Java (Handles HTTP requests, orchestration, and business logic without heavy framework overhead).
- **Extraction & Utility:** Pure Java Cryptography Architecture (JCA) (Stateless extraction of raw binary .pem files into mapped domain models).
- **Data Access Layer:** PostgreSQL (Relational storage utilizing native JSONB for flexible domain tags and strict schema enforcement for cryptographic facts).
- **AI/Heuristic Layer:** Python (Consumes backend data to provide contextual RAG analysis).

2.2 The Ingestion & Audit Pipeline (Current State)

The platform decouples data ingestion from compliance evaluation to ensure a complete audit trail.

- **The Ingestion:** Certificates are parsed and saved to the certificates table with an initial status of UNAUDITED.
- **The Queue:** A scheduled background worker queries PostgreSQL for certificates where the last_audited_at timestamp is older than 24 hours.
- **The Audit:** Certificates are processed by a deterministic Java validation engine. The result updates the certificates state and appends a new immutable record to the audit_logs table.

Why this works now: This two-table polling architecture requires no complex event queues (e.g., Kafka). It is transactional, safe, and provides the exact time-series data the Python AI layer needs to learn and perform historical analysis.

3. Scaling Strategy: Prototype to Enterprise

As the platform scales to monitor hundreds of thousands of certificates across global infrastructures, the current architecture will encounter specific bottlenecks. The following strategies dictate how the system must evolve.

3.1 Resolving the Polling Bottleneck (The "Thundering Herd")

- **The Constraint:** At scale, querying the database for all certificates older than 24 hours will pull massive datasets into Java's memory simultaneously, causing CPU spikes and lock contention.
- **The Enterprise Shift:** The background worker must implement **Pagination and Chunking**. The database query will be modified to fetch small, manageable batches (e.g., 1,000 rows per transaction), process them, and release memory before requesting the next batch.

3.2 Managing Data Explosion (Table Bloat)

- **The Constraint:** Logging an audit event for 500,000 certificates daily results in 15 million rows per month in the audit_logs table, severely degrading read performance over time.
- **The Enterprise Shift:**
    1. **State-Change Logging:** The system will transition to logging only when an AuditReport mutates (e.g., transitioning from COMPLIANT to EXPIRING_SOON), rather than logging redundant daily confirmations.
    2. **Table Partitioning:** PostgreSQL native partitioning will be enabled to split audit_logs by month, allowing instant archiving of historical data without locking the primary tables.

3.3 Eliminating the Revocation Blind Spot

- **The Constraint:** A 24-hour polling cycle creates a critical delay. If a certificate is compromised immediately after its daily audit, the dashboard will display a false COMPLIANT status until the next cycle.
- **The Enterprise Shift:** The polling mechanism will be replaced (or supplemented) by an **Event-Driven Architecture (EDA)**. The system will expose webhook endpoints to receive real-time push events from external Certificate Authorities or Certificate Transparency (CT) logs, triggering an instantaneous, targeted audit of the compromised certificate.